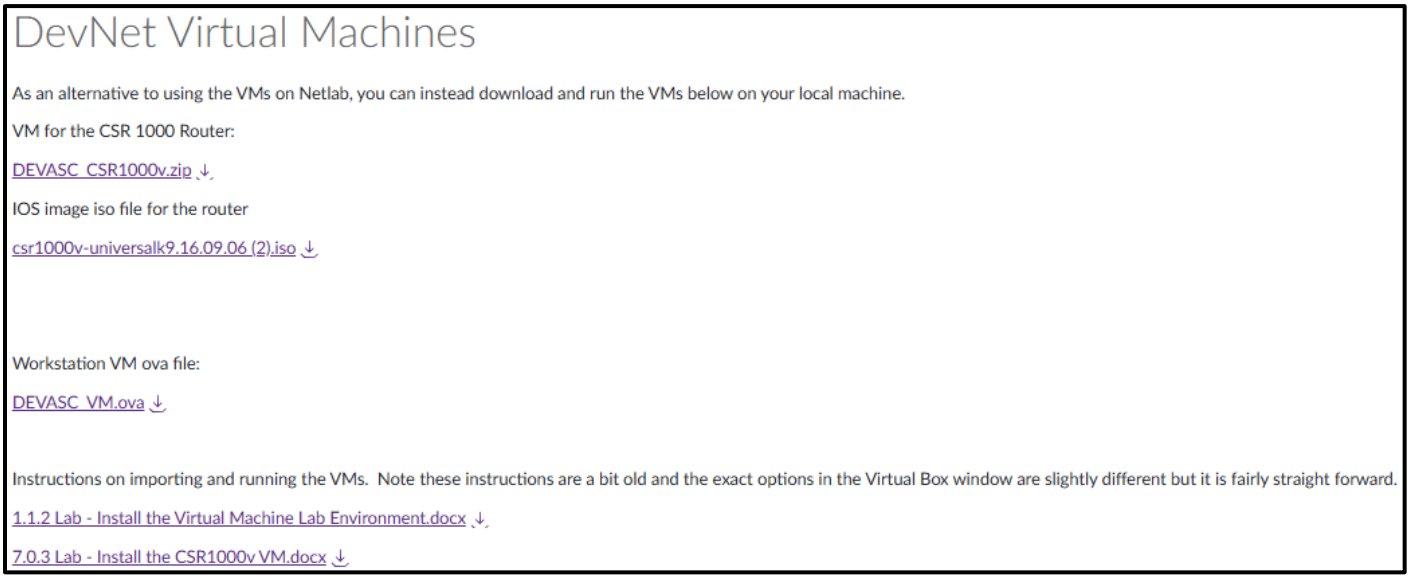
Student Lab Manual – Introduction to Model-Driven Network Automation Using NETCONF

Lab Overview

This lab may be completed using **NDG NETLAB+** or a local **DEVASC-LABVM** & **CSR1000v** VM Working in Tandem (PREFERRED METHOD). If using the DEVASC-LABVM & CSR1000v VM option, see Canvas for instructions on installing Virtual Machines & Routers on the Home tab (see screenshots below), as permitted by Dr. Pickard. Connectivity and credential availability may vary by environment.



Background / Scenario

Network automation has traditionally relied on sending CLI commands to devices through scripts. While effective, this approach often produces unstructured output, making validation and error handling more difficult.

Modern network platforms support model-driven automation, which allows applications to work with devices using structured data models instead of raw text. NETCONF is one of these protocols, which enables

configuration data to be retrieved and modified using structured formats like XML. This approach improves predictability and reduces the risk of unintended changes.

In this lab, you will use Python and NETCONF to safely retrieve and modify interface configuration on a Cisco IOS XE (Virtual Router) device. The emphasis is on automation safety, validation, and structured data rather than speed or scale.

Lab Objectives

After completing this lab, you will be able to:

- Explain the difference between CLI-based automation and model-driven automation
- Use NETCONF to retrieve structured configuration data
- Apply a safe interface-level configuration change using NETCONF
- Verify configuration changes by examining post-change device state
- Describe how structured automation improves reliability and validation

---

Estimated Completion Time

45–75 minutes

Required Resources

- NDG NETLAB+ environment
- Cisco IOS XE router with NETCONF enabled
- DEVASC Linux VM
- Python 3 (any supported version)
- ncclient Python library
- Provided Python automation scripts

Note: This lab does not require external Internet access. All required tools and device access are provided within the NETLAB+ environment.

Additional Requirement for Local (VirtualBox) Execution

If you are completing this lab using the **DEVASC VirtualBox VM** (instead of NDG NETLAB+), an additional virtual machine is required: **CSR1000v virtual router**. Installation instructions and reference screenshots are provided on Canvas.

The DEVASC VM includes the automation tools required for this lab, such as Python and the **ncclient** library, but it does **not** include a Cisco IOS XE device. Since NETCONF automation requires a reachable, NETCONF-enabled router, the CSR1000v must be added to the VirtualBox environment when completing the lab locally.

Note: Depending on current system availability, some students may find the local VirtualBox setup to be more reliable than the NDG NETLAB+ environment. Refer to Canvas announcements and discussion posts for any known connectivity considerations when choosing an environment.

To complete this lab locally, students must have:

- A Cisco IOS XE or CSR1000v virtual router (preferred)
- Running in VirtualBox
- Reachable from the DEVASC VM
- NETCONF enabled (`netconf-yang`)
- TCP port 830 open

Safety Guidelines

To ensure safe operation of this lab:

- Do not modify routing protocols, VLANs, or production services
- All configuration changes are limited to interface descriptions or administrative state
- No credentials are hardcoded in automation scripts
- All changes are reversible
- Follow steps in order and stop if unexpected output occurs

Network Topology

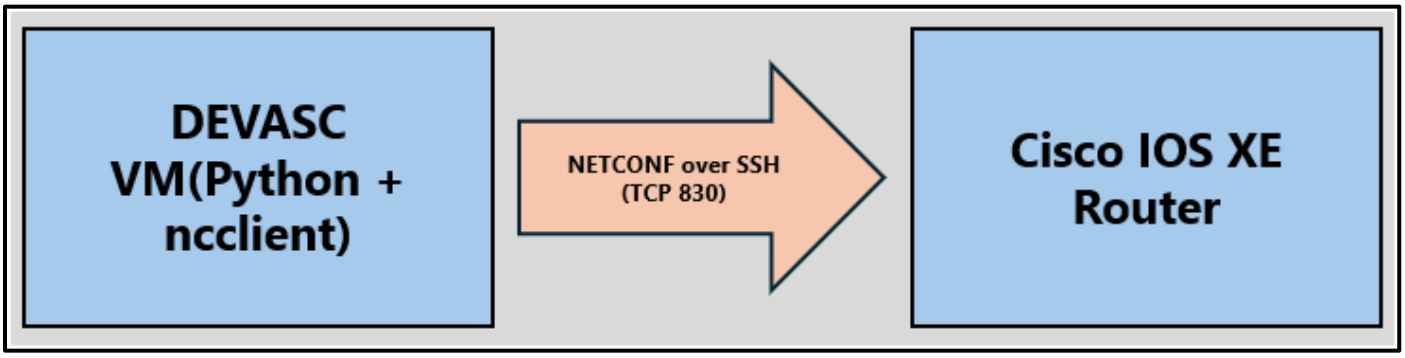
This lab uses a simple, single-device topology. The Cisco IOS XE device is provided either through the NDG NETLAB+ DevNet environment or as a local IOS XE / CSR1000v virtual router when the lab is completed using the DEVASC VirtualBox VM.

Automation tasks are performed from the DEVASC Linux virtual machine using Python and the ncclient library. The DEVASC VM communicates directly with the Cisco IOS XE device using NETCONF over SSH (TCP port 830) to retrieve and modify structured configuration data.

No additional network devices or inter-device dependencies are introduced. This intentionally limited topology is designed to keep the focus on model-driven automation concepts, safety, and validation rather than on network scale or protocol interaction. Figure 1 below illustrates the logical topology used in this lab:

Figure 1

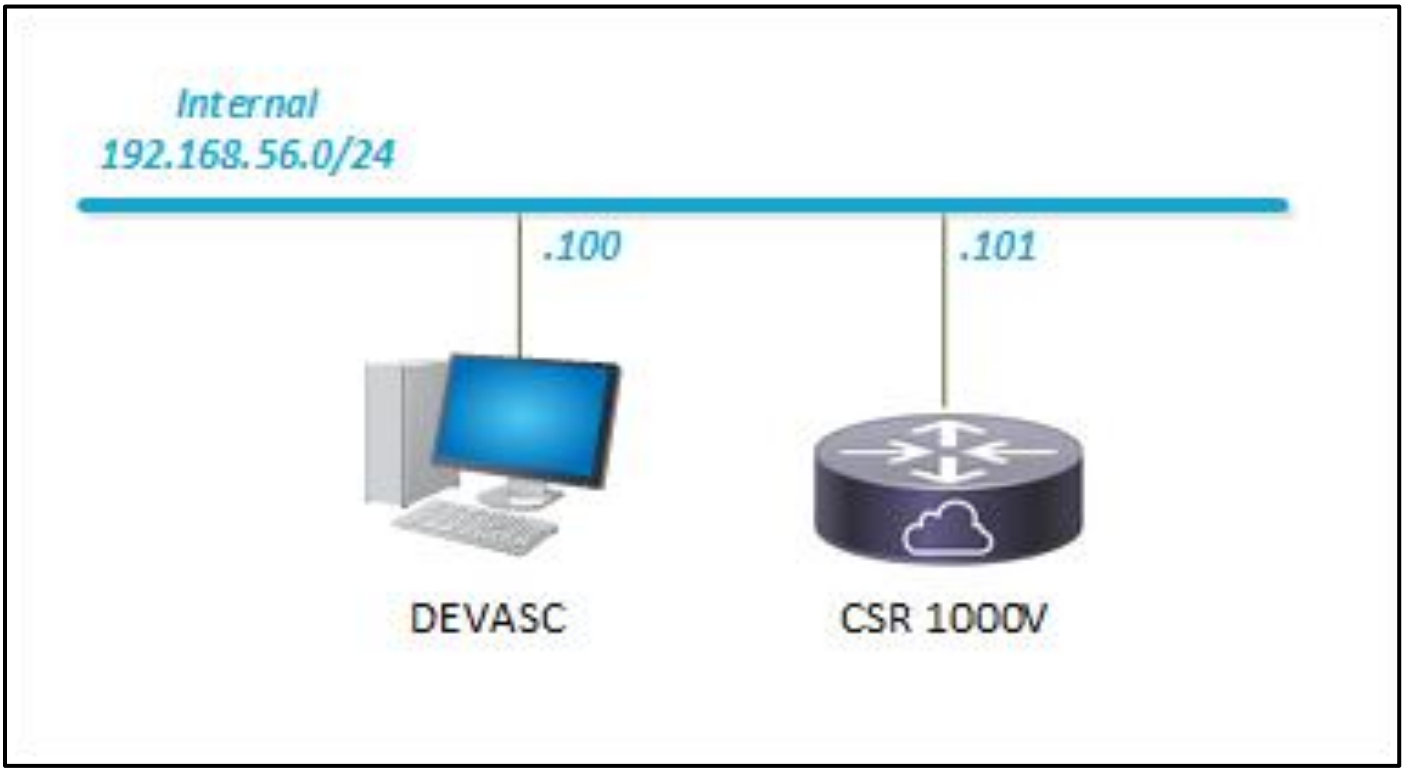
NETCONF-Based Automation Topology: [DEVASC-LABVM] ---- NETCONF (SSH / TCP 830) ----> [IOS XE / CSR1000v]



If using the NDG NETLAB+ Environment, the topology will look as such:

Figure 2

NDG NETLAB+ topology (includes CSR1000v)



Pre-Lab / Setup

Step 1: Preparing the CSR1000v (virtual router) and accessing the CSR1000v Router Console

1.) Power on the CSR1000v virtual machine in VirtualBox.

2.) Click inside the VirtualBox console window so that keyboard input is captured.

3.) Allow the router to complete its boot process. System messages related to cryptographic services and Plug-and-Play (PnP) may appear; these messages are normal.

4.) When the router displays the following prompt, it is ready for user input:

```
CSR1kv>
```

5.) Enter privileged EXEC mode:

```
Enable
```

6.) Verify that the prompt changes to:

```
CSR1kv#
```

7.) Enter global configuration mode:

```
configure terminal
```

8.) Verify that the prompt changes to:

```
CSR1kv(config)#
```

9.) Set the router hostname:

```
hostname CSR1000v
```

10.) Verify that the prompt updates to:

```
CSR1000v(config)#
```

Step 2: Configuring the CSR1000v Management Interface

1.) Verify that you are in global configuration mode and that the router prompt displays:

```
CSR1000v(config)#
```

2.) Enter the management interface configuration mode:

```
interface GigabitEthernet1
```

3.) Verify that the prompt changes to:

```
CSR1000v(config-if)#
```

4.) Assign an IP address to the management interface (static or via DHCP, depending on environment):

If configuring a static IP address:

```
ip address 192.168.56.10 255.255.255.0
```

OR (if DHCP is used by the environment, and preferred for this lab):

```
ip address dhcp
```

5.) Enable the management interface:

```
no shutdown
```

6.) Exit interface configuration mode:

```
exit
```

7.) Verify that the router prompt returns to:

```
CSR1000v(config)#
```

8.) Verify the interface status and IP assignment:

```
do show ip interface brief
```

9.) Confirm that GigabitEthernet1:

- Has an IP address assigned
- Displays a status of up/up

Part 1: Verify the Automation Environment

In this part, you will confirm that the Python automation environment and NETCONF connectivity are ready before performing device automation tasks.

1.) Log into NDG NETLAB+ and launch the DevNet pod OR power on the DEVASC-LABVM and CSR1000v virtual machines in VirtualBox (Preferred Method)

2.) Open the DEVASC virtual machine.

3.) Open a terminal window.

4.) Navigate to the directory containing the provided lab files (for example):

Verification

- The script executes without errors.
- A NETCONF session to the IOS XE device is established successfully.
- No configuration changes are applied to the device during this step. If errors occur, verify that Python and ncclient are installed correctly.

If errors occur, verify that:

- Python 3 (any supported version) is installed and functioning correctly.
- The `ncclient` library is installed.
- The CSR1000v router is reachable, and NETCONF is enabled.

Part 2: Retrieve Interface Configuration Using NETCONF

In this part, you will retrieve interface configuration data from the Cisco IOS XE device using NETCONF. No configuration changes are made during this step.

Step 1: Create the NETCONF Retrieval Script

1.) From the DEVASC virtual machine, open a terminal window.

2.) Navigate to the directory containing the lab files (for example):

```
cd ~/labs/devnet-src/python
```

3.) Create a new Python file named:

```
nano netconf_get_interfaces.py
```

4.) Type the following script into the file exactly as shown:

```
"""
NETCONF Interface Retrieval Script
This script retrieves interface configuration data from a Cisco IOS XE device
using a NETCONF <get> operation. No configuration changes are made.
"""

from ncclient import manager

# Device connection details
DEVICE_IP = "192.168.56.102"
USERNAME = "netconfuser"
PASSWORD = "Cisco123!"

# NETCONF filter to retrieve interface configuration data
```

```
interface_filter = """
<filter>
  <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  </interfaces>
</filter>
"""

# Establish NETCONF session
with manager.connect(
    host=DEVICE_IP,
    port=830,
    username=USERNAME,
    password=PASSWORD,
    hostkey_verify=False,
    device_params={"name": "iosxe"}
) as netconf_manager:

    print("NETCONF session established successfully.\n")
    print("Retrieving interface configuration data...\n")

    # Perform NETCONF <get> operation
    result = netconf_manager.get(interface_filter)

    # Display structured XML output
    print(result.xml)
```

5.) Save and exit the editor:

- Press Ctrl + O, then Enter
- Press Ctrl + X

Step 2: Review the NETCONF Retrieval Logic

Before executing the script, review its contents and identify how it:

- Establishes a NETCONF session using the ncclient library
- Uses a NETCONF `<get>` operation to retrieve data
- Requests interface configuration information using a YANG-based filter

Do **not** modify the script during this step.

Step 3: Execute the NETCONF Retrieval Script

1.) Run the script:

```
python3 netconf_get_interfaces.py
```

2.) Observe the output returned by the device. You should see something like this:

```
<data>
  <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>GigabitEthernet1</name>
      <description>VBox</description>
      <enabled>true</enabled>
      <ipv4>...</ipv4>
    </interface>
  </interfaces>
</data>
```

Verification

To successfully complete this part:

- XML-formatted configuration data is displayed.
- Interface elements such as interface name, IP addressing, or administrative state appear in structured XML output.
- No configuration changes are applied to the device.

This step demonstrates how NETCONF provides structured, machine-readable data rather than unstructured CLI text.

Expected Outcome

- A NETCONF session is successfully established. You should see:

  ```
  NETCONF session established.
  Retrieving interface configuration...
  ```

- Interface configuration data is retrieved using a NETCONF <get> operation.
- The device configuration remains unchanged.

Part 3: Apply a Safe Interface Configuration Change

In this part, you will use a NETCONF <edit-config> operation to modify device configuration.

Step 1: Modify the NETCONF Script

1.) Open the existing NETCONF retrieval script:

```
nano netconf_get_interfaces.py
```

2.) Modify the script to apply one safe configuration change using a NETCONF `<edit-config>` operation.

For this lab, update the interface description of `GigabitEthernet1`.

Step 2: Replace the <get> Operation with <edit-config>

Modify the NETCONF script used in **Part 2** (netconf_get_interfaces.py) so that it sends the following configuration to the device using a NETCONF <edit-config> operation:

```
<config>
  <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>GigabitEthernet1</name>
      <description>Configured via NETCONF</description>
    </interface>
  </interfaces>
</config>
```

Step 3: Save and Execute the Script

1.) Save the script and exit the text editor.

2.) Run the script:

```
python3 netconf_get_interfaces.py
```

Verification

- NETCONF returns a successful reply (`<ok/>`).
- No error messages or partial failures occur.
- The device remains reachable after the change.
- Stop after receiving the `<ok/>` response.
- You will verify the configuration change in **Part 4**.

You should see:

```
<rpc-reply ...>
  <ok/>
</rpc-reply>
```

Part 4: Verify Post-Change Device State

Verification is a critical component of reliable network automation. In this part, you will confirm that the configuration change applied in **Part 3** is present and that no unintended configuration changes occurred.

Step 1: Restore and Run the NETCONF Retrieval Script

In **Part 3**, the NETCONF script was modified to perform a write operation (<edit-config>).
Before verification, ensure that the script performs a read-only NETCONF <get> operation.

1.) From the DEVASC virtual machine, open a terminal window.

2.) Navigate to the directory containing the lab files (for example):

```
cd ~/labs/devnet-src/python
```

3.) Open the NETCONF retrieval script:

```
nano netconf_get_interfaces.py
```

4.) Ensure the script contains the following NETCONF retrieval logic:

```
from ncclient import manager

DEVICE_IP = "192.168.56.102"
USERNAME = "netconfuser"
PASSWORD = "Cisco123!"

interface_filter = """
<filter>
  <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  </interfaces>
</filter>
"""

with manager.connect(
    host=DEVICE_IP,
    port=830,
    username=USERNAME,
    password=PASSWORD,
    hostkey_verify=False,
    device_params={"name": "iosxe"}
) as netconf_manager:

    print("NETCONF session established.")
    print("Retrieving interface configuration...\n")

    result = netconf_manager.get(interface_filter)
    print(result.xml)
```

Note: This script uses a NETCONF <get> operation and does not modify device configuration.

5.) Save and exit the editor:
    Press Ctrl + O, then Enter
    Press Ctrl + X
6.) Run the NETCONF retrieval script:
    python3 netconf_get_interfaces.py

Step 2: Observe and Compare the Output

1.) Examine the XML output returned by the device.\
2.) Compare the current output to the interface configuration retrieved in Part 2.
3.) Confirm that the interface description of GigabitEthernet1 reflects the change applied in **Part 3**.

For example:

```
<description>Configured via NETCONF</description>
```

Verification

To successfully complete this part:

- The updated interface description is visible in the structured XML output.
- The change matches exactly what was configured in **Part 3**.
- No other interface parameters (such as IP address or administrative state) were modified.
- No errors are returned by the NETCONF operation.

This confirms that the automation change was applied successfully and that the device is in the intended state.

Validation Summary

You have successfully:

- Established NETCONF connectivity
- Retrieved structured configuration data
- Applied a controlled configuration change
- Verified the device state after the change

This workflow demonstrates a complete and safe automation process.

Review Questions

1. Why is NETCONF output easier to validate than CLI output?

   Hint: Think about the difference between:

   - Reading configuration as structured XML fields (as you saw with NETCONF), versus
   - Reading configuration as unstructured text (as would be returned by CLI commands).

   Consider how easy it was in this lab to confirm one specific change (the interface description) using NETCONF output.

   Answer:

2. What risks exist when configuration changes are only partially applied?

   Hint: Think about what could happen if a configuration change succeeds only *part of the way*. Consider how traditional CLI command sequences might leave a device in an inconsistent or unexpected state if an error occurs mid-change, and compare that to how NETCONF applies changes.

   Answer:

3. How does structured automation improve reliability?

   Hint: Think about what could happen if a configuration change succeeds only *part of the way*. Consider how traditional CLI command sequences might leave a device in an inconsistent or unexpected state if an error occurs mid-change, and compare that to how NETCONF applies changes.

   Answer:

Reflection

In your own words, explain how NETCONF-based automation improves reliability compared to traditional CLI-based automation. Consider structured data, validation, and error handling in your answer.

Answer: